

React Interview Questions for Mid-Level Developers

Components and Lifecycle

Q1. What are the differences between class components and functional components in React?

Answer: Class components are ES6 classes that extend `React.Component` and can hold state (via `this.state`) and lifecycle methods like `componentDidMount` or `shouldComponentUpdate`. They require using `this` to access props and state, and you update state with `this.setState`. Functional components, on the other hand, are plain JavaScript functions that accept props as arguments and return JSX. Initially, functional components were stateless, but with the introduction of **Hooks** in React 16.8, they can manage state and side effects too (using `useState`, `useEffect`, etc.), eliminating the need for most class components. Functional components are generally preferred now because they are simpler, don't use `this`, and make code easier to read and test. In summary:

- **Class Components:** Use ES6 class syntax, have lifecycle methods, manage state with `this.state` and `setState`, and were traditionally used for stateful logic.
- **Functional Components:** Use function syntax, **no built-in lifecycle methods** (use Hooks instead), manage state with Hooks like `useState`, and tend to result in cleaner and more concise code. After Hooks, functional components can do everything classes can, making them the common choice for new code.

Q2. Explain the React component lifecycle phases and methods (in class components).

Answer: React class components go through **three main lifecycle phases: Mounting, Updating, and Unmounting**. Each phase has special lifecycle methods you can override to run code at specific times

1 2 :

- **Mounting phase:** Occurs when the component is being created and inserted into the DOM.
Important methods:
 - `constructor(props)` : Initializes state and binds methods (runs before mounting).
 - `render()` : Returns the JSX to render (called during mounting and updates).
 - `componentDidMount()` : Invoked after the component is mounted in the DOM. It's commonly used for initiating data fetches or setting up subscriptions 3 .
- **Updating phase:** Occurs when the component's props or state change, causing a re-render.
Methods include:
 - `shouldComponentUpdate(nextProps, nextState)` : Allows you to prevent unnecessary re-renders by returning `false` if an update isn't needed.

- `componentDidUpdate(prevProps, prevState)`: Runs after the component has updated (after re-render). Useful for reacting to prop/state changes (e.g., triggering an API call when a prop changes) ². If you perform side-effects here, you should often wrap them in a condition to avoid infinite loops.
- **Unmounting phase**: Occurs when the component is being removed from the DOM. The key method:
 - `componentWillUnmount()`: Invoked right before the component is destroyed. Use it to clean up resources such as event listeners, intervals, or subscriptions to prevent memory leaks ⁴.

Additionally, there are **error handling methods** (often considered part of lifecycle): `componentDidCatch` and `getDerivedStateFromError` for error boundaries (see next question). Modern React also has newer lifecycle methods like `getDerivedStateFromProps(props, state)` (which runs before rendering on updates to adjust state based on prop changes) and the deprecated `componentWillReceiveProps`. However, with Hooks and functional components, these class lifecycle methods are largely replaced by the `useEffect` Hook for managing side effects in various phases.

Q3. What are error boundaries in React, and how do you implement one?

Answer: An **error boundary** is a special React component that **catches JavaScript errors** anywhere in its child component tree, logs those errors, and displays a fallback UI instead of crashing the whole app. Error boundaries catch errors during rendering, in lifecycle methods, and in constructors of the whole subtree below them ⁵. To implement an error boundary, you typically create a class component that defines two lifecycle methods: `static getDerivedStateFromError(error)` and `componentDidCatch(error, info)`. For example:

```
class MyErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }
  static getDerivedStateFromError(error) {
    // Update state so the next render shows fallback UI
    return { hasError: true };
  }
  componentDidCatch(error, info) {
    // You can log the error or send it to an analytics service
    console.error("Caught error:", error, info);
  }
  render() {
    if (this.state.hasError) {
      return <h2>Something went wrong.</h2>; // Fallback UI
    }
    return this.props.children;
  }
}
```

You would wrap this around parts of your app: `<MyErrorBoundary><SomeComponent/></MyErrorBoundary>`. If an error occurs in `SomeComponent` or its children, the error boundary will catch it and render the fallback UI. **Note:** Error boundaries catch errors in the **DOM rendering** and lifecycle, but not in event handlers (you must handle those manually with try/catch). Also, error boundaries **must be class components** (as of React 18), because Hooks don't have an equivalent yet.

Q4. What is a Higher-Order Component (HOC) in React and when would you use one?

Answer: A **Higher-Order Component (HOC)** is a design pattern in React where you create a function that takes a component and returns a **new component** with enhanced capabilities. In simpler terms, an HOC is a function **that wraps a component with additional props or functionality**, allowing you to reuse logic across multiple components ⁶. The typical signature is: `const EnhancedComponent = higherOrderComponent(WrappedComponent)`.

For example, if you have common logic for fetching data or handling subscriptions needed by several components, you can create an HOC that provides that data as props to any component:

```
function withUserData(WrappedComponent) {
  return class extends React.Component {
    state = { user: null };
    componentDidMount() {
      fetch('/api/current_user')
        .then(res => res.json())
        .then(userData => this.setState({ user: userData }));
    }
    render() {
      // pass down fetched user data as a prop
      return <WrappedComponent user={this.state.user} {...this.props} />;
    }
  };
}
```

Now you can use `withUserData` to enhance any component that needs user data:

```
const ProfileWithUser = withUserData(ProfileComponent);
```

When to use HOCs: Before Hooks, HOCs (and Render Props) were common for reusing stateful logic (like connecting to a store or handling subscriptions). Now, many use cases for HOCs can be handled with custom Hooks in functional components. However, HOCs are still useful for concerns like Redux's `connect` (which is an HOC) to inject state and dispatch into components, or for cross-cutting concerns such as authentication checks, theming, etc. Just be mindful that multiple nested HOCs can make the component tree harder to debug (the "wrapper hell" issue).

Hooks

Q5. What are React Hooks and why were they introduced?

Answer: **Hooks** are functions provided by React that let you "hook into" React features (like state and lifecycle) from functional components. They were introduced in React 16.8 to allow functional

components to use state and other React features, removing the need for many class components. Hooks solve several problems: - They avoid the confusion of `this` context in classes. - They let you reuse stateful logic without restructuring your component hierarchy (through custom hooks, instead of patterns like HOCs). - They result in cleaner, more concise code by splitting logic based on concerns (e.g., separate hooks for separate pieces of logic) rather than lifecycle methods which can contain unrelated code.

Common built-in Hooks include `useState` (for managing state), `useEffect` (for side effects), `useContext`, `useReducer`, `useMemo`, `useCallback`, `useRef`, etc. Hooks significantly changed how we write React code – now most components are functional components using Hooks. One important thing: **Hooks have rules** – they can only be called at the top level of a functional component (not inside loops or conditions) and only within React function components or custom hooks ⁷.

Q6. How does the `useState` Hook work?

Answer: `useState` is a Hook that lets functional components have state. You call `const [state, setState] = useState(initialValue)`. This returns a **state variable** (initialized to `initialValue`) and a **setter function** to update that state. Key points about `useState`: - You can use multiple `useState` calls in one component for separate state variables (each call creates independent piece of state). - Updating state via the setter (e.g., `setState(newValue)`) will schedule a re-render of the component. React then re-renders and the `state` variable now holds the new value. - The state update is **asynchronous** and may be batched – you won't immediately see the new value in the current function execution. If you need to use the updated value to compute the next state, use the functional update form: `setState(prev => /* compute using prev */)`. - Unlike class `setState`, the state set with `useState` **does not merge** automatically (it replaces the value). If your state is an object, you must spread the previous state manually if you want to update part of it.

Example:

```
const [count, setCount] = useState(0);
...
// increment count
setCount(prevCount => prevCount + 1);
```

This will update `count` and trigger a re-render. Using the functional form `prevCount => prevCount + 1` ensures you get the correct previous value even if updates are batched.

Q7. How is `useEffect` different from lifecycle methods like `componentDidMount` or `componentDidUpdate`?

Answer: The `useEffect` Hook lets functional components perform **side effects** (like data fetching, subscriptions, or manually manipulating the DOM) after rendering. It essentially combines the roles of several lifecycle methods: - Without dependencies, `useEffect` runs **after every render** (like `componentDidUpdate` for all updates). - With an empty dependency array `[]`, `useEffect` runs only once after the initial mount (equivalent to `componentDidMount`). - If you return a cleanup function from the effect, that function runs before the component unmounts or before the effect re-runs (like `componentWillUnmount`).

Key differences from class lifecycle: - In classes, you have separate methods for different times (mount, update, unmount). With `useEffect`, one function can handle all those by adjusting the dependency array. - `useEffect` runs asynchronously after rendering and after the DOM has been updated. By default, effects are deferred until after painting for performance. - Unlike `componentDidUpdate`, an effect can be configured to run only for certain state/prop changes by listing them as dependencies. React will skip the effect if those values haven't changed between renders. - If no dependency array is provided, the effect runs after every render (which can be expensive if not intended). - There's also `useLayoutEffect`, which is similar but fires synchronously after DOM mutations (like `componentDidMount/DidUpdate` but blocking visual update, used when you need to measure DOM elements).

For example, to mimic `componentDidMount`, you'd do:

```
useEffect(() => {  
  // effect logic (runs once on mount)  
}, []); // empty dependencies
```

And to mimic `componentWillUnmount`, you return a cleanup:

```
useEffect(() => {  
  const subscription = API.subscribe(...);  
  return () => {  
    // cleanup  
    subscription.unsubscribe();  
  };  
}, [/* dependencies */]);
```

One must be careful: if you specify some dependency incorrectly or omit it, the effect might not run when expected or run too often. Also, **if you don't pass a dependency array at all, the effect will run on every render**, which is rarely desired and can lead to performance issues or even infinite loops if the effect updates state unconditionally ⁸ ⁹.

Q8. What are the rules of Hooks and why must they be followed?

Answer: The **Rules of Hooks** are two important constraints that ensure Hooks work correctly: 1. **Only call Hooks at the top level** – don't call Hooks inside loops, conditions, or nested functions. Always use them at the top level of your React function. This rule ensures that Hooks are called in the same order every time a component renders, so that React can correctly keep track of each hook's state between renders. 2. **Only call Hooks from React functions** – you should only call Hooks from within React functional components or from custom Hook functions. You should not call Hooks in regular JavaScript functions, class methods, or outside a component.

These rules are enforced by the linter plugin `eslint-plugin-react-hooks` to catch mistakes. Breaking these rules can lead to bugs because React wouldn't be able to properly associate Hook calls with state (since the order might change). As an example, **do not** put a `useState` or `useEffect` inside an `if` block – instead, structure your Hook usage so that it's always called in the same sequence on every render ⁷.

Q9. What is `useRef` used for?

Answer: The `useRef` Hook provides a way to hold a **mutable value** across renders *without* causing a re-render when it changes. It returns an object with a single property `{ current: ... }`. Common use cases for `useRef`:

- **Accessing DOM elements:** You can pass a ref object to a JSX element (e.g., `<input ref={myRef} />`) and then access `myRef.current` to get the actual DOM node. This is similar to using `React.createRef` in class components.
- **Storing mutable values that persist between renders:** For example, you can keep a mutable variable that doesn't trigger re-renders. This is useful for things like holding a timeout ID, keeping track of previous value, or any value you want to remember without re-rendering on change. E.g. `const countRef = useRef(0);` then `countRef.current` can be updated in an event handler without causing the component to re-render.
- **Implementing instance variables:** In classes, you might use `this.someVar` to store data that persists. In functional components, `useRef` can do that.

One important note: if you update `ref.current`, React **does not re-run the component**. This is why refs shouldn't be used to manage data that's used for rendering (those should go in state). Refs are for external or instance-like data. Also, `useRef` can be used to **persist values** through re-renders (unlike a normal variable that resets on each render).

Q10. When would you use `useReducer` instead of `useState`?

Answer: `useReducer` is a Hook that's an alternative to `useState` for managing state logic. It's particularly useful when you have **complex state logic** that involves multiple sub-values or when the next state depends on the previous state. With `useReducer`, you define a reducer function (similar to Redux reducers) that takes the current state and an action, and returns a new state. You then get the state and a `dispatch` function from `useReducer`.

Scenarios to prefer `useReducer`:

- When you have **multiple related state values** that you want to manage together or more predictably. For example, managing a form with many input fields and complex update logic – a reducer can handle updating the nested state in one place.
- When the state transitions have a clear cause and effect (e.g., actions like `"INCREMENT"`, `"RESET"`, etc.). This can make the code more readable than having many `useState` calls or complex state objects updated via one `setState`.
- When you want to **centralize state logic** outside the component (you can define the reducer separately) or even reuse it across components.
- It's also useful for **performance** in some cases – if you pass `dispatch` down to child components, it avoids passing setter functions for each piece of state and can reduce re-renders in complex trees (since `dispatch` identity is stable).

For example:

```
const initialCount = 0;
function reducer(state, action) {
  switch(action.type) {
    case 'increment': return { count: state.count + 1 };
    case 'decrement': return { count: state.count - 1 };
    case 'reset': return { count: 0 };
    default: throw new Error();
  }
}
const [state, dispatch] = useReducer(reducer, { count: initialCount });
```

Here, calling `dispatch({ type: 'increment' })` updates the state based on the reducer logic. This approach scales well for complex state, and it mirrors Redux patterns at a component level. For simple state or a small number of state variables, `useState` is usually sufficient (and more straightforward).

Q11. What are `useMemo` and `useCallback`, and how do they help with performance?

Answer: Both `useMemo` and `useCallback` are Hooks that help optimize performance by **memoizing** values or functions so that they only change when their dependencies change, rather than on every render:

- `useMemo(fn, deps)`: Returns a **memoized value**. You pass it a function that computes a value and an array of dependencies. It will only re-run that computation when one of the dependencies changes. This is useful for expensive calculations or deriving data that you don't want to redo on every render. For example, `const derivedData = useMemo(() => heavyCompute(data), [data]);` ensures `heavyCompute` runs only when `data` changes, otherwise it reuses the last result.
- `useCallback(fn, deps)`: Returns a **memoized version of a callback function**. It's essentially like `useMemo` but specifically for functions. `useCallback` is useful when passing callbacks to child components to prevent unnecessary re-renders. For instance, if you pass a callback prop down, the child might re-render each parent render if the function is recreated every time. Wrapping it in `useCallback` with appropriate dependencies ensures the same function instance is used until deps change.

When to use them: These hooks are not for every situation – only use them when a component's re-render or a function recreation is causing a real performance issue (measured via profiling). Overusing them can add complexity. But some common use cases: - Memoizing expensive computations (filtering a large list, complex calculations) so they don't run on each render. - Preventing re-render of pure child components by ensuring props like functions or objects have stable references unless needed. (E.g., using `useCallback` for an event handler that you pass to a child wrapped in `React.memo`.) - Caching derived data (with `useMemo`) that is used in rendering to avoid recalculation.

In summary, `useMemo` returns a memoized **value**, `useCallback` returns a memoized **function**. Both should list dependencies that determine when the memo should update. If the dependencies don't change, the previously cached result is used, saving recomputation or preventing child updates.

Q12. What is a custom Hook, and can you give an example use case?

Answer: A **custom Hook** is a reusable function that uses one or more React Hooks inside of it to encapsulate and share stateful logic. Custom hooks are a way to abstract component logic into a separate function so it can be reused easily across multiple components, following the DRY principle.

- A custom hook is just a **JavaScript function whose name starts with `use`** (to obey the Hooks rule). It can call other hooks (`useState`, `useEffect`, etc.) and either return some state and/or functions.

For example, imagine you want multiple components to have access to the current window size (and update when the window resizes). You can write a custom hook `useWindowSize`:

```
function useWindowSize() {
  const [size, setSize] = useState({ width: window.innerWidth, height:
window.innerHeight });
  useEffect(() => {
    function handleResize() {
      setSize({ width: window.innerWidth, height: window.innerHeight });
    }
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);
  return size;
}
```

Now any component can use this hook:

```
function MyComponent() {
  const windowSize = useWindowSize();
  return <p>Width: {windowSize.width}, Height: {windowSize.height}</p>;
}
```

This component will automatically re-render when the window is resized, because `useWindowSize` encapsulated that behavior. Custom hooks are great for abstracting away concerns like data fetching (`useFetch` hook that handles fetching logic and returns loading/error/data), form handling, subscription to services, etc. They promote code reuse without the drawbacks of HOCs or render props, and keep components focused on presentation rather than logic.

State Management (State, Props, and Context)

Q13. What are props in React and how are they different from state?

Answer: Props (short for properties) are inputs to components. They are passed from a parent component down to a child component, much like function arguments. Props are **read-only** – a component should never modify its own props. They are used to **configure a component** or pass data for rendering ¹⁰. For example, `<Button color="red" text="Click me" />` passes two props to the `Button` component.

State, in contrast, is data that is managed *within* a component (though it can be passed down as props to children). State is **mutable** (can change over time), and when it changes, it triggers a re-render of the component. Each component can manage its own state (using `useState` in functional components or `this.state` in class components). State is used for data that can change due to user interactions, network responses, etc.

Key differences: - **Ownership:** Props are owned by the parent and get passed in. State is owned by the component itself. - **Mutability:** Props are immutable in the child (the parent can change what it passes). State is managed and updated by the component (using `setState` or hooks). - **Use cases:** Props allow parent components to communicate with children. State allows components to manage dynamic data internally. For example, a parent might pass a list of items as a prop to a child list component (child just renders them), while the child might manage an internal state like which item is selected.

Remember that in React's one-way data flow, **data flows down** via props, and if a child needs to communicate up, it usually does so via callbacks passed down as props.

Q14. What is "lifting state up" and why would you do it?

Answer: Lifting state up is the pattern of moving state upward to the closest common ancestor of components that need to share that state ¹¹ ¹². If two sibling components or otherwise distant components need to use or synchronize the same piece of state, you can lift that state to a parent component that encompasses both, and then pass it down via props.

For example, if Component A and Component B both need to know the selected value of a dropdown that lives in A, you can lift the state for the selection to their parent. The parent holds the selection in state and passes it to A (to control the dropdown value) and to B (so B can do something with it). Component A also receives a callback prop from the parent to update the selection in the parent's state when the dropdown changes.

Why do this? **To avoid duplicate state and out-of-sync data.** If each component managed its own copy of the state, they could diverge. Lifting state up ensures there's a "single source of truth" for that data, and both components reflect the same state. It also helps eliminate **prop drilling** in some cases (passing props through many layers) by consolidating state higher and only passing what's needed.

The downside is it can make the parent component larger or more complex, but it's often a necessary step for consistent state. React's official docs often show this pattern when multiple components need to reflect the same changing data (like a temperature input in Celsius and Fahrenheit example).

Q15. What is prop drilling and how can it be avoided?

Answer: Prop drilling is a situation in React where you have to pass props down through multiple intermediary components that do not actually need those props, just so that a deeply nested child can use them. This often happens when some state is held high up (or in a parent), but far-down components need to use or update it – you end up passing props at every level in between, which can make those intermediate components messy and dependent on props they don't really care about.

For example, if component A -> B -> C -> D, and D needs a piece of data that A has in state, you might have to pass that data as a prop from A to B, B to C, and C to D, even though B and C don't use it themselves.

Avoiding prop drilling: The main tools to avoid prop drilling are the **Context API** or using a state management library like **Redux**: - **React Context:** Context provides a way to share values (state) globally across the component tree without explicitly passing props through every level. You create a context and use a Provider to wrap the tree, then any deeply nested component can use `useContext` (or Context consumer) to access the value directly ¹³. This is great for things like theming, current user, locale, or any data that many components need. - **Redux or other global stores:** These allow you to store data in a global store and connect components directly to that store, so intermediate components don't need to pass the props along. - Also, sometimes **lifting state up** strategically can reduce drilling by moving state to a common ancestor that's closer to all consumers, as described above.

Using context is the built-in way to solve prop drilling for moderate needs, whereas Redux is often used in larger apps for complex global state. Keep in mind that context is best for relatively static or

infrequently changing data (or things like current theme), because excessive re-renders via context changes can also be an issue if not used carefully (context will propagate updates to all consumers).

Q16. What are controlled and uncontrolled components in React forms?

Answer: In the context of form inputs in React, **controlled components** are form elements (like `<input>`, `<textarea>`, `<select>`) where the **source of truth** for their value is **React state**. An **uncontrolled component** is one that maintains its own internal state (the DOM itself keeps track of the value) and you query it via refs when needed, rather than synchronizing it with React state on every change ¹⁴ ¹⁵.

- In a **controlled component**, you give the input a `value` prop that is tied to state, and an `onChange` handler that updates that state. For example:

```
const [name, setName] = useState("");
return <input value={name} onChange={e => setName(e.target.value)} />;
```

Here React state (`name`) controls the input's displayed value at all times. If `name` doesn't change, the input won't change. This gives you full control – you can enforce validation, manipulate input (like forcing uppercase), and respond to changes immediately. Controlled components are usually preferred for complex forms because they make the form data accessible and manipulable by React (e.g., easily preventing invalid input, disabling submit if errors, etc.). They are also easier to test, since you can simulate state and events to see how the component reacts ¹⁴.

- In an **uncontrolled component**, you do not provide a `value` (or you might use `defaultValue` initially). The DOM itself keeps track of the input value as the user types. To get the value, you typically use a ref. For example:

```
const inputRef = useRef();
<input type="text" ref={inputRef} />;
// Later, when you need the value:
const currentVal = inputRef.current.value;
```

Uncontrolled components are simpler to set up for basic use cases because you don't need state or `onChange` handlers. They can be useful for quick forms or when integrating with non-React code. However, you have less insight into what the user is typing at any given moment until you read the value from the DOM, and validation can be trickier.

Summary: Controlled components are driven by React state (the recommended approach for most scenarios, giving you fine control), whereas uncontrolled components rely on the browser DOM state. Controlled components provide more control and enable instant validation but involve more code. Uncontrolled components are more like traditional HTML form inputs (simpler, but with logic deferred until you need the value) ¹⁴ ¹⁵.

Q17. What is the Context API and how do you use it?

Answer: The **Context API** in React is a built-in feature that allows for sharing data across components without having to pass props manually at every level. Context is often described as providing **global-ish** or **app-wide** variables that various components can access, which is useful for things like theme, user info, locale, or any data that many parts of the app need.

Using Context: 1. You create a context with `React.createContext(defaultValue)`. This gives you a Context object. 2. You wrap a section of your component tree with a **Provider** for that context: `<MyContext.Provider value={someValue}> ... </MyContext.Provider>`. This value is now available to any nested component within that tree. 3. Any component in that tree can access the context value by using `useContext(MyContext)` Hook in a functional component, or `<MyContext.Consumer>` in class or older patterns. With Hooks, it's straightforward:

```
const value = useContext(MyContext);
```

This gives you the nearest Provider's `value`.

For example, if you have a ThemeContext:

```
const ThemeContext = React.createContext('light'); // default 'light'
function App() {
  const [theme, setTheme] = useState('dark');
  return (
    <ThemeContext.Provider value={theme}>
      <Toolbar />
    </ThemeContext.Provider>
  );
}
function Toolbar() {
  return <ThemeButton />; // somewhere deep
}
function ThemeButton() {
  const theme = useContext(ThemeContext);
  return <button className={theme}>My theme is {theme}</button>;
}
```

In this example, `ThemeButton` can read the current theme without the `Toolbar` or other intermediate components needing to receive it as a prop.

When to use Context: When many components at different levels need the same data. Context can prevent prop drilling (as discussed). However, context is not a replacement for all state management. Overusing context for every piece of data might cause lots of re-renders (if context value changes often) or make components less reusable (tightly coupling them to a context).

Also, context updates will re-render all components that consume that context. For fine-grained global state updates, sometimes libraries like Redux or Zustand are more efficient. A common pattern is to use

context in moderation, for things like theme or auth user info, and possibly to provide a Redux store or other global store to the tree (Redux uses context under the hood in its `<Provider>`).

Global State and Redux

Q18. What is Redux and what problem does it solve?

Answer: **Redux** is a popular state management library often used with React (though it can be used with any UI framework). It provides a predictable way to manage **global state** in an application. The main problems Redux addresses are: - **State sharing and consistency:** In a large app, many components might need to access or update certain state. Instead of passing props through many levels or having duplicate inconsistent state, Redux puts the state in a single global store so it's a "single source of truth". - **Predictability:** Redux has a strict structure – state is read-only and can only be changed by dispatching actions that are handled by pure functions called reducers. This makes state transitions explicit and easier to debug or test. - **Debugging and DevTools:** Redux's architecture allows time-travel debugging (you can replay actions to see how state changed over time) with tools like the Redux DevTools. This is very useful in development.

Key concepts in Redux: - **Store:** A single JavaScript object holding the whole state of your app. - **Action:** A plain object that describes a state change, usually with a `type` field and some payload. Actions are dispatched (sent) to the store. - **Reducer:** A pure function `(state, action) => newState` that takes the current state and an action and returns a new state. The reducer defines how actions transform the state. - **Dispatch:** A function to send actions to the store. - **Subscriber (View):** Components subscribe to the store; when state updates, they get the new state and re-render.

In a React-Redux app, you typically use the **React-Redux** bindings: - Wrap your app with `<Provider store={store}>` so that all components can access the store. - Use either the older `connect` function or newer Hooks (`useSelector` to read state, `useDispatch` to dispatch actions) in your components to interact with Redux.

By using Redux, you get a clear data flow: **components dispatch actions -> actions are processed by reducers -> new state is generated and stored -> components re-render based on new state** ¹⁶. This one-way data flow is very predictable (often called **Flux architecture**).

Redux is especially useful in mid to large applications where many parts of the UI need to reflect the same changing data (e.g., a user's cart in an e-commerce site, which can be updated from different places like product list or cart page). It can be overkill for very small apps, though. Redux also has concepts of **middleware** for handling side effects (like async API calls via `redux-thunk`, etc.), which further structure how state changes happen.

Q19. How do you integrate Redux with a React application?

Answer: To use Redux with React, you typically use the official **React-Redux** library. The steps are: 1. **Create a Redux store** using `createStore` (or `configureStore` if using Redux Toolkit) with your root reducer (and possibly middleware). 2. **Provide the store** to your React app by wrapping your app's root component with `<Provider store={store}>` from `react-redux`. This makes the Redux store available to any nested component that needs it. 3. In your React components, **access the store** in one of two ways: - Using the **Hooks API:** `useSelector(selectorFn)` to read state and `useDispatch()` to get the dispatch function for sending actions. - Using the **connect HOC:** The older way is to define `mapStateToProps` and `mapDispatchToProps` and use `connect(mapStateToProps,`

`mapDispatch)(YourComponent)` ¹⁷ ¹⁸. This injects state and dispatch props into the component. For example, `connect` might inject a `count` prop from state and an `increment()` prop that dispatches an action. 4. Define your **actions** and **reducers** that describe how the state can be updated when actions occur. 5. Components will dispatch actions (e.g., on a button click, `dispatch({ type: 'INCREMENT' })`) and Redux's reducer(s) will create a new state. The React-Redux integration automatically causes any components using `useSelector` or connected via `connect` to re-render with the new state.

A simple example:

```
// store.js
import { createStore } from 'redux';
const initialState = { count: 0 };
function counterReducer(state = initialState, action) {
  switch(action.type) {
    case 'INCREMENT': return { count: state.count + 1 };
    case 'DECREMENT': return { count: state.count - 1 };
    default: return state;
  }
}
export const store = createStore(counterReducer);
```

```
// index.js
import { Provider } from 'react-redux';
import { store } from './store';
ReactDOM.render(
  <Provider store={store}>
    <App />
  </Provider>,
  document.getElementById('root')
);
```

```
// CounterComponent.js (using hooks)
import { useSelector, useDispatch } from 'react-redux';
function Counter() {
  const count = useSelector(state => state.count);
  const dispatch = useDispatch();
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => dispatch({ type: 'INCREMENT' })}>+1</button>
    </div>
  );
}
```

This would display the count and increment it in the global store when the button is clicked.

Key integration points: The Provider at the top, and then usage of either hooks or `connect` in components. Also note if you use Redux Toolkit, setting up the store often involves `configureStore` which adds some default middleware (like for immutability checks).

Q20. When should you use Redux instead of React's local state or Context API?

Answer: Redux is a powerful tool, but it's not needed for every application. Here are considerations for choosing Redux: - **App Size and Complexity:** If your application is small or medium and state is mostly local to individual components or only a few pieces need sharing, React's built-in state and Context might be sufficient. Redux shines in larger apps with complex interactions and a lot of shared state (or when you need to trace changes in a predictable way). - **Multiple Components Needing the Same State:** If many distant components need to read and update the same state, and that state is frequently updated, Redux can provide a structured approach. Context can do this too, but Redux provides more tooling (like middleware, dev tools, time-travel debugging). - **Predictability and Debugging:** Redux forces state updates to be pure and centralized, which can make debugging easier – all changes are through actions and you can log them or use the dev tools. If you find it hard to track who changes state or you have bugs from inconsistent state, Redux might help enforce discipline. - **Scaling Team/Project:** In large teams, having a single predictable state flow (Flux pattern) can help coordination. Redux also often comes with conventions that teams adopt (like all state changes being logged, or using Redux Toolkit to standardize patterns). - **Alternatives (Context API or other libraries):** Context API is good for passing down data but lacks built-in middleware and devtools. Also, updating context causes *all* consumers to re-render, so performance might suffer if you put a lot of rapidly changing state in context. Redux, by connecting only the components that need the data, can be more selective in updates. There are also simpler libraries (like Zustand, MobX, or recoil) – some prefer those for simpler usage if Redux is too much boilerplate.

In summary, **use Redux** when your state management starts getting unwieldy with just React state or context – e.g., lots of props drilling or complex update logic – and you want a more predictable, centralized solution. If your app state is simple or mostly local, Redux might be overkill (it adds complexity of its own). A common mid-level understanding: "Use Redux for global app state that many things need (especially if to be modified in various places), but keep UI-local state (like form inputs, toggles, etc.) in component state or context" ¹⁹ ²⁰ .

Q21. What are Redux middleware and can you give examples?

Answer: Middleware in Redux are extra layers that sit between dispatching an action and the moment it reaches the reducer. They intercept actions and can perform tasks, then pass actions along (or modify them, or dispatch other actions). Middleware is a way to extend Redux with custom functionality, especially for things that are not pure synchronous operations.

Common uses of middleware: - **Handling asynchronous actions:** Redux by itself only handles plain object actions. Middleware like **redux-thunk** allows action creators to return a function (thunk) instead of an action object. That function can perform async calls (like fetch data) and then dispatch real actions when ready. Example:

```
// with thunk middleware
function fetchUsers() {
  return function(dispatch, getState) {
    dispatch({ type: "FETCH_USERS_REQUEST" });
    fetch('/api/users')
```

```

    .then(res => res.json())
    .then(
      data => dispatch({ type: "FETCH_USERS_SUCCESS", payload: data }),
      error => dispatch({ type: "FETCH_USERS_ERROR", error })
    );
  };
}

```

- **Logging or analytics:** A logging middleware can log every action and state after it's processed (useful for debugging). - **Error reporting:** Middleware could catch errors in action processing and report them. - **Other side effects:** e.g., routing (react-router's older integration with Redux could intercept navigation actions), or things like socket messages.

Examples of popular middleware: - **redux-thunk:** As mentioned, allows functions (thunks) for async logic. - **redux-saga:** Uses generator functions to handle complex async workflows in a more testable way via sagas (separate from the component dispatch flow). - **redux-logger:** Logs actions to console for development. - **redux-promise:** Allows you to dispatch promises as actions (it will dispatch resolved value as an action). - **redux-toolkit** (RTK Query): Internally provides middleware for caching queries, etc.

Technically, a middleware is a function with signature `(({ dispatch, getState }) => next => action)`. Inside it, you can do something with the action and call `next(action)` to pass it to the next middleware or reducers. They are applied when creating the Redux store (e.g., `applyMiddleware(thunk, logger)`).

In summary, middleware enrich Redux by handling side effects and non-pure operations, keeping reducers pure and the overall flow more structured ²¹.

Q22. What is Redux Toolkit and how is it different from traditional Redux?

Answer: Redux Toolkit (RTK) is the official, recommended way to write Redux logic now. It's essentially a wrapper around Redux that provides **sane defaults and abstractions** to make Redux development faster and less error-prone. It addresses some common complaints about Redux (boilerplate, complexity) by: - Providing helper functions to create slices of state with reducers and actions (`createSlice`), so you don't have to write action type constants, action creators, and switch statements manually for each slice of state. - Including **Immer** library under the hood, so you can write mutating syntax in reducers (e.g., `state.value += 1`) and it will actually produce an immutable update behind the scenes. - Setting up good default middleware like `redux-thunk`, and `devTools` integration by default. - It also includes utilities like `createAsyncThunk` for common async patterns, and `RTK Query` for data fetching caching out of the box.

Differences from traditional Redux: - RTK is **opinionated** (in a helpful way): it suggests a project structure and provides one big `configureStore` call to set up store with middleware and `devtools`. - Much **less boilerplate**: for example,

```

const slice = createSlice({
  name: 'counter',
  initialState: 0,
  reducers: {
    increment: state => state + 1,
  }
})

```

```
    decrement: state => state - 1
  }
});
export const { increment, decrement } = slice.actions;
export default slice.reducer;
```

This generates action creators for increment/decrement and a reducer internally handling those actions. You don't write switch statements or separate files for types. - It encourages using the immutable update pattern with Immer (so you can write more natural code). - If you start a new React project and add Redux via RTK, you'll follow patterns that avoid common mistakes (like mutating state, or forgetting to remove boilerplate).

In short, **Redux Toolkit** makes Redux simpler, more powerful (through included packages), and less verbose ²². Mid-level developers should be familiar with RTK because modern codebases likely use it instead of raw Redux createStore.

Performance Optimization

Q23. What is the virtual DOM and how does React use it for performance?

Answer: The **virtual DOM (VDOM)** is an in-memory, lightweight representation of the real DOM. In React, when state or props change, React creates a new virtual DOM tree (a set of JavaScript objects describing the UI) and then **diffs it** against the previous virtual DOM tree to see what has changed ²³. This process is called **reconciliation**. After diffing, React knows exactly which parts of the actual DOM need to be updated and can apply those minimal changes.

Why this helps performance: Directly updating the real DOM is relatively slow. By batching changes and minimizing direct DOM manipulation, React avoids unnecessary reflows/repaints. The virtual DOM diff is efficient (using algorithms and keys to detect changes), and updating only what's needed means fewer DOM operations, which generally results in faster, smoother UI updates ²⁴.

Benefits:

- It makes UI updates more **declarative**: you describe what the UI should look like in the new state, and React handles the optimal way to update the real DOM to match that. - It can batch multiple state changes into a single update (especially with React's batching in event handlers or using Concurrent features), leading to fewer layout calculations. - The virtual DOM approach enables features like React's **Concurrent Mode** (in React 18) which can prepare updates in the background without blocking the main thread.

Downsides:

- There is some overhead in maintaining the virtual DOM and doing the diffing computations (it's not free). For very simple scenarios or extremely dynamic updates (like animations via canvas or extremely frequent DOM changes), a manual approach could in theory be faster. But in most apps, React's efficiency gains outweigh the overhead ²⁵. - Developers need to be mindful of giving stable keys for list items to help the diffing algorithm (discussed in the next question).

Overall, the virtual DOM is one of React's core features that enables its performance and declarative programming model.

Q24. Why is using keys in lists important in React?

Answer: **Keys** are a special prop you should include when rendering lists of elements in React (e.g., when using `array.map` to produce elements). A key is a string or number that uniquely identifies each element in the list relative to its siblings. For example:

```
{items.map(item => <ListItem key={item.id} value={item} />)}
```

Using keys is important because it helps React identify which items have changed, been added, or removed. During the reconciliation process, React uses keys to match elements between the old and new virtual DOM lists ²⁶. If you use unique, stable keys: - React can efficiently reorder or update only the changed elements. For instance, if an item in the middle of the list is removed, keys let React know which one, and it can remove that DOM node and keep others as is, rather than re-rendering the entire list. - Keys should be **unique** among siblings and **stable** (not change between renders, unless that element is truly going to be different). Good keys are usually an ID coming from your data.

What if you don't use keys or use poor keys (like array indices)?

- If no keys are provided, React will warn you and fall back to using indexes, which can lead to inefficient updates. - Using **array index as a key** is a common mistake. It works for static lists that never reorder, but if you later insert or reorder items, using indices can cause issues: the key may identify a different item before vs. after the change, causing React to mistakenly reuse DOM elements for different data. This can lead to strange bugs (e.g., input losing focus or retaining the wrong value because React reused an element) ²⁷. - Without proper keys, React will re-render all items in a list whenever one changes, because it can't reliably tell which item corresponds to which previous item.

In short, keys make list diffing more efficient and correct. They are a hint to React: "this element's identity is this key". Always use a stable unique key for items in a list (like an ID from data, or if no ID, something stable like a combination of strings that represent the item).

Q25. What are some ways to optimize the performance of a React application?

Answer: Performance optimization in React often involves minimizing unnecessary renders and reducing the workload of each render. Here are several strategies and techniques:

- **Avoid Unnecessary Re-renders:**
- **PureComponent/React.memo:** Use `React.PureComponent` (for classes) or wrap functional components with `React.memo` to do a shallow prop comparison and skip re-rendering if props haven't changed ²⁸. This is useful for components that render often with the same props.
- **shouldComponentUpdate:** In class components, implement `shouldComponentUpdate` to manually prevent re-renders when you know props/state changes don't require an update.
- **Avoid changing reference values needlessly:** For example, avoid recreating new objects or functions in parent on every render unless needed (or use `useCallback` / `useMemo` to memoize them). Changing a prop from a parent that's an object literal defined in JSX each time will cause child to think props changed.
- **Keys in lists:** as discussed above, proper keys help React efficiently update lists instead of re-rendering them entirely.
- **Code-Splitting and Lazy Loading:**

- Use React's `React.lazy` and `Suspense` to **lazy-load** components that aren't needed immediately (for example, routes that aren't shown on initial load). This can drastically improve initial load time by splitting your bundle into smaller chunks loaded on demand.
- Similarly, dynamic import for large libraries only when needed (e.g., importing a heavy datepicker library when a date field is focused).

- **Memoization of expensive calculations:**

- Use `useMemo` to cache results of expensive calculations or derived data so that you don't recompute on every render unless inputs change.
- Likewise, `useCallback` to avoid recreating functions (which could trigger expensive re-renders in optimized children).

- **Windowing/Virtualization for large lists:**

- If you have long lists or tables (hundreds or thousands of items), use a windowing technique (via libraries like **react-window** or **react-virtualized**) to render only a portion of the list that is visible. This way, you aren't paying the cost to render thousands of off-screen DOM nodes.

- **Optimize Context usage:**

- If using React Context for global data, be mindful that any change in context value will re-render all consumers. Sometimes splitting context into multiple contexts for different pieces of state can isolate updates. Or use selective memoization in consumer components.

- **Browser Performance Tweaks:**

- Use CSS transitions/animations instead of frequent state changes for animations when possible.
- Debounce or throttle expensive operations (like a search input that filters a list – don't update on every keystroke in real-time, or throttle window resize updates).
- For canvas or heavy graphics in React, sometimes using direct DOM or ref manipulation might be needed for performance, or leveraging WebGL.

- **Production optimizations:**

- Make sure to use the production build of React (which removes dev warnings and is smaller/faster).
- Enable tree-shaking by importing only what you need (for example, `lodash.submodules` or `date-fns` instead of `moment`, etc., to reduce bundle size).
- Use a performance monitoring tool or the React DevTools Profiler to pinpoint slow components and optimize them specifically.

In practice, you would measure performance (using DevTools Profiler) and apply these optimizations where they have impact. For mid-level developers, understanding `React.memo`, keys, and basic memoization and lazy loading are key. Also remember that readability is often more important than micro-optimizing – don't prematurely optimize unless there's a known issue.

Q26. What is React's Concurrent Mode (or concurrent features) and how do they improve performance?

Answer: React's **Concurrent Mode** (now referred to as "concurrent features" in React 18, as full Concurrent Mode isn't a toggle but certain features like `startTransition`, Suspense improvements, etc.) is a set of updates to React's rendering engine (React Fiber) that allows it to be more **interruptible and efficient** in how it renders updates.

In traditional React (pre-concurrent), once an update started rendering, it was synchronous and could block the browser for a noticeable time if the UI was complex. Concurrent features allow React to: - **Pause, abort, or delay work** on lower-priority updates so that higher priority updates (like user input) can be handled first. This means smoother apps because a large render (say, updating a big list) can be spread over multiple frames or paused if the user does something high priority like typing. - **Split rendering into multiple chunks**. React Fiber (the reconciliation algorithm since React 16) breaks work into units. Concurrent rendering leverages that to not do everything in one go. - `startTransition`: In React 18, you can mark state updates as "transitions" (low priority) so that state update will be processed concurrently, yielding to more urgent updates. For example, when switching a route, you might show a spinner immediately (urgent state update) and load the new page content under a transition. React will keep the old UI until the new content is ready and render it without blocking the user's ability to interact with the spinner or other UI. - `Suspense` **for data fetching**: With concurrent features, Suspense can let you start rendering UI while deferring some parts until data is loaded, without blocking the entire render.

From an interview perspective: Concurrent Mode improves perceived performance and responsiveness by making rendering more **asynchronous** and controllable. It's a complex topic, but essentially it allows React to avoid blocking the main thread for long periods and to coordinate multiple state updates more gracefully ²⁹ ³⁰ .

It's worth noting that these features are largely automatic – you opt-in by using things like `Suspense` for data, `useDeferredValue` or `startTransition`, etc. As a mid-level developer, you should be aware that React 18 can do things like batch updates by default (which is part of concurrent behavior) and that marking updates as transitions can help avoid locking up the UI. Also, certain older methods (like `forceUpdate` or some lifecycle patterns) might not play well with concurrency.

In summary, concurrent features make React **"interruptible"** and smarter about scheduling rendering, leading to smoother user experiences in complex apps.

Routing (React Router)

Q27. What is React Router and why is it used in React applications?

Answer: **React Router** is a widely used library that enables routing and navigation in React applications. Since React apps are typically single-page applications (SPA) where the page doesn't fully reload on navigation, React Router allows developers to define **multiple routes (URLs)** within their app and map them to React components. Key points: - It lets you declare routes in your JSX (usually wrapped in a router provider) such that certain components render based on the URL path. This gives users a multi-page app experience (with browser history, bookmarkable URLs) while still being an SPA under the hood. - **Why use it:** React by itself doesn't have an opinion about routing. React Router provides a standard pattern for handling navigation state in the URL. It keeps the UI in sync with the URL and allows for features like browser back/forward navigation, route parameters, nested routes, etc., with

ease ³¹ . - It supports **client-side routing** – meaning when the user clicks a link, the router intercepts it, updates the URL using History API, and renders a different component, all without a full server page load. This makes navigation fast and fluid. - React Router also has features like **dynamic routing** (routes can use parameters like `/user/:id`), **nested routes** (components containing sub-routes), **redirects**, and more, which are essential for structuring a complex app.

In short, React Router is used to manage navigation in a React application, handling what content to display for each URL, and to provide a good user experience with deep linking and browser navigation controls in SPAs.

Q28. How do you define routes and links in React Router?

Answer: In React Router (v6, the latest major version), you typically do the following: - **Set up the Router:** At the top level of your app (usually in `index.js` or `App.js`), wrap your app in a router component, most commonly `<BrowserRouter>`:

```
import { BrowserRouter } from 'react-router-dom';
ReactDOM.render(
  <BrowserRouter>
    <App />
  </BrowserRouter>,
  document.getElementById('root')
);
```

`<BrowserRouter>` uses the HTML5 history API to keep the UI in sync with the URL (suitable for modern web servers that handle client-side routes). Alternatively, `<HashRouter>` can be used if you can't configure server-side to support client routes – it uses a hash (`#`) portion in URLs to simulate route state.

• **Define Routes:** Inside your app (often in App component), use the `<Routes>` container and multiple `<Route>` elements:

```
import { Routes, Route } from 'react-router-dom';
function App() {
  return (
    <div>
      <Navbar />
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/about" element={<AboutPage />} />
        <Route path="/users/:id" element={<UserProfile />} />
        <Route path="*" element={<NotFoundPage />} /> { /* catch-all for
404 */}
      </Routes>
    </div>
  );
}
```

Each `<Route>` specifies a `path` and an `element` (the component to render when the path matches). The `path` can include parameters like `:id`, which will match any value in that segment and make it available for the component to use (via hooks like `useParams`). The `*` route is a wildcard to catch any unmatched URL.

- **Links/Nav:** Use the `<Link>` component to create navigation links without causing full page reloads. Example:

```
import { Link } from 'react-router-dom';
<Link to="/about">About Us</Link>
```

This renders as a normal `<a>` tag, but intercepts clicks to navigate within the SPA. For more imperative navigation (like redirecting after a form submission or on a button click), you can use the `useNavigate()` hook:

```
import { useNavigate } from 'react-router-dom';
const navigate = useNavigate();
// ...
navigate('/login');
```

This will programmatically change the route.

- **Nested Routes:** In v6, you can define nested `<Route>` inside another route by using an `Outlet`. For example:

```
<Route path="/dashboard" element={<DashboardLayout />}>
  <Route index element={<DashboardHome />} />
  <Route path="analytics" element={<AnalyticsPage />} />
</Route>
```

In `DashboardLayout`, you would include an `<Outlet />` component where child routes render. This allows parts of the page to change while keeping common layout for the section.

In earlier versions (v5), you used `<Switch>` instead of `<Routes>` and component/render props on `Route`. But in v6, the above is the approach.

So, defining routes is declarative: wrap in a `Router` (`BrowserRouter` or `HashRouter`), use `Routes/Route` for mapping URL paths to components, and use `Link` (or `NavLink` for styling active link) for navigation. This covers basic navigation setup in React Router.

Q29. How do you get access to URL parameters or query strings in a React component using React Router?

Answer: To access route parameters (the parts of the URL that are dynamic, like `:id` in a route path), you can use the **Hooks** provided by React Router v6: - `useParams()`: This hook returns an object of key-value pairs of the dynamic params in the URL. For example, if you have a route `<Route path="/users/:id/details/:section" ...>`, then inside the component rendered by this route:

```
import { useParams } from 'react-router-dom';
const { id, section } = useParams();
```

`id` and `section` will be strings corresponding to the actual values in the URL. You can then use those (often to fetch data specific to that id, etc.).

For query string parameters (like `?sort=asc&page=2`), `useParams` does not cover those, because query strings are not part of the route pattern. Instead: - `useLocation()`: This hook gives you the location object, which includes the full `search` string. For example:

```
import { useLocation } from 'react-router-dom';
const location = useLocation();
console.log(location.search); // e.g. "?sort=asc&page=2"
```

You can parse `location.search` using the browser's `URLSearchParams`:

```
const query = new URLSearchParams(location.search);
const sort = query.get('sort'); // "asc"
const page = Number(query.get('page')); // 2
```

Alternatively, some prefer to use `useSearchParams()` hook (in React Router v6) which provides a nicer API:

```
import { useSearchParams } from 'react-router-dom';
const [searchParams, setSearchParams] = useSearchParams();
const sort = searchParams.get('sort');
```

`useSearchParams` gives a mutable `URLSearchParams` that you can also update easily (e.g., `setSearchParams({ sort: 'desc' })` to change the query string).

In older versions (v5), you might have used `match.params` from route props, or `useRouteMatch`, but in v6, `useParams` is the straightforward way. And for query, previously you'd manually parse location search as above.

Example usage: If a route is defined as `/product/:productId`, in the `ProductDetail` component:

```
const { productId } = useParams();
// likely trigger a useEffect to fetch product detail by productId
```

This gives access to that segment.

So, `useParams` for path params and `useLocation` or `useSearchParams` for query string. These hooks require that the component is inside a Router (which is usually the case if it's rendered via a `<Route>`).

Q30. What is the difference between `<BrowserRouter>` and `<HashRouter>`? When might you use one over the other?

Answer: Both `<BrowserRouter>` and `<HashRouter>` are router implementations provided by React Router, but they handle URLs differently:

- **BrowserRouter:** Uses the **HTML5 History API** (`pushState`, `popState` events) to manipulate the URL. The URLs it generates look like normal, clean paths (e.g., `http://myapp.com/about`). This requires the server to be configured to respond to these routes (for example, by serving the same `index.html` for any route, falling back to client routing). When using `BrowserRouter`, if you refresh the page or directly visit a nested URL, the server needs to handle that path (usually by serving the app) or you get a 404 from the server. In a production setup, you typically configure server-side to redirect requests to `index.html` or use a catch-all.
- **HashRouter:** Uses the URL **hash (#) portion** to keep track of the "route". For example, a route would look like `http://myapp.com/#!/about`. Everything after the `#` isn't sent to the server (browsers don't send the hashfragment to the server), so the server will always just see the path before `#` (which is usually just `/` for the base page). This means you don't need special server configuration – the client app will parse the part after `#` and route accordingly. `HashRouter` is useful if you cannot or don't want to configure the server, such as if you're deploying to GitHub Pages or some static host where you can't easily redirect 404s to `index.html`.

When to use: - Use **BrowserRouter** in most modern web applications where you have control over the server setup. It produces nicer-looking URLs (no hash) and is generally the standard for production SPAs. It also allows the use of browser history in a more conventional way (though `HashRouter` also syncs to history, but differently). - Use **HashRouter** if you're constrained by hosting (like older GitHub Pages support) or building something that will be opened via `file://` protocol, or any situation where refreshes on routes cannot be handled by a server. It's also sometimes used for client-side apps that are embedded or need to work without server-side support for routing.

In summary, the difference is in how routing information is stored in the URL (History API vs hash). `BrowserRouter` requires server support for client routes, `HashRouter` does not. As a mid-level dev, you'd primarily use `BrowserRouter`, but it's good to know `HashRouter` as a fallback.

Q31. How can you implement protected routes or route guards in React Router?

Answer: Protecting routes (e.g., requiring authentication to view certain pages) isn't built into React Router as a first-class feature, but it can be implemented by utilizing the capabilities of Router. There are a couple of approaches:

- **Wrapper Route Component (Higher-Order Route):** You can create a component that checks authentication and either redirects or renders the component. For example:

```
// A component that wraps the Route element
function PrivateRoute({ element: Component, ...rest }) {
```

```
const isAuthenticated = // ... determine if user is authenticated
return (
  <Route
    {...rest}
    element={ isAuthenticated ? <Component /> : <Navigate to="/login" /> }
  />
);
}
```

Then use `<PrivateRoute path="/dashboard" element={Dashboard} />` instead of a normal Route. This will redirect to `/login` if not authenticated (using `<Navigate>` which is React Router v6's component for redirects).

- **Use Hooks in the component itself:** Another method is to do the check inside the component using `useNavigate`. For example:

```
function Dashboard() {
  const isAuthenticated = // check auth from context or redux
  const navigate = useNavigate();
  useEffect(() => {
    if (!isAuthenticated) {
      navigate('/login', { replace: true });
    }
  }, [isAuthenticated]);
  if (!isAuthenticated) {
    return null; // or a spinner while redirecting
  }
  return <SecureDashboardContent />;
}
```

This way, if the user is not auth, the effect will immediately redirect them away.

- **Outlet context (for nested routes):** If using nested routes for an entire section that's protected, you could have a parent route check auth and only render an `<Outlet />` (child routes) if authenticated, otherwise `<Navigate>` away.

Important bits: - You need to have a way to know if the user is authenticated (this might come from a context or Redux store that holds auth info like a token or user object). - Use `<Navigate to="/login" />` in v6 (or `Redirect` in v5) to perform the actual redirect.

Example PrivateRoute (v6 updated approach):

If one uses element approach:

```
const RequireAuth = ({ children }) => {
  let auth = useAuth(); // custom hook that checks auth context
  let location = useLocation();
  if (!auth.user) {
    // redirect to login, but save current location to come back after login
  }
}
```

```

    return <Navigate to="/login" state={{ from: location }} replace />;
  }
  return children;
};

// Usage in routes:
<Routes>
  <Route path="/dashboard" element={
    <RequireAuth>
      <Dashboard />
    </RequireAuth>
  } />
  <Route path="/login" element={<LoginPage />} />
</Routes>

```

In this example, if not authenticated, it navigates to login, and after login, you could navigate back to `state.from` so user goes where they originally intended.

This pattern is essentially a guard around certain routes. The key is using either a wrapper or inline logic to decide whether to render the protected component or redirect. React Router leaves that decision to you, which is flexible.

Q32. What is code-splitting and how can you implement it in a React app (particularly with React Router)?

Answer: Code-splitting is the practice of breaking your JavaScript bundle into smaller chunks that can be loaded on demand. In a React application, this often means not loading all components and libraries upfront, but instead loading them as needed, for example when a user navigates to a certain route or performs a certain action. The benefit is improved initial load time and overall performance – users download less JS for the initial screen and load other parts when needed.

In React, code-splitting is commonly done with **dynamic** `import()` and React's **lazy/Suspense**: - React provides `React.lazy()` to wrap a dynamic import for a component. For example:

```
const SettingsPage = React.lazy(() => import('./SettingsPage'));
```

This will split out `SettingsPage` (and its dependencies) into a separate chunk that loads when it's rendered the first time. - You must render lazy components within a `<Suspense fallback={...}>` component which defines what to show (e.g., a loading spinner) while the code is loading:

```

import { Suspense } from 'react';
<Suspense fallback=<div>Loading...</div>>
  <SettingsPage />
</Suspense>

```

- With React Router, you can integrate this by lazy-loading route components:

```
const SettingsPage = React.lazy(() => import('./SettingsPage'));
// in your routes
<Route path="/settings" element={
  <Suspense fallback={<Spinner />}>
    <SettingsPage />
  </Suspense>
} />
```

So when the user goes to `/settings`, the code for `SettingsPage` is fetched and loaded at that time. Until it's loaded, a `Spinner` can be shown.

Alternatively, you can use route-based splitting outside of Router by splitting your routes configuration. Some setups use a wrapper to load components as needed, but the above is the straightforward way with v6.

For non-React Router scenario, you can also dynamic-import components on an event, or pieces of libraries. For example, if you have a heavy chart library only used in one component, you can import it inside that component's effect or when needed instead of at top of file.

Real example: If using Create React App or webpack, dynamic imports will automatically create separate bundle files. If not using `Suspense` (older approach), you might manually use a library like `Loadable Components` or `react-loadable`, but with modern React, `Suspense` for code splitting is the way.

In summary, **code-splitting** in React is typically achieved with `React.lazy` and `Suspense`, especially to split on route boundaries. This ensures users only download the code necessary for the routes they visit, improving performance by loading less code upfront.

Testing

Q33. What is Jest and how is it used in testing React applications?

Answer: **Jest** is a popular JavaScript testing framework maintained by Facebook. It's commonly used for testing React applications (in fact, Create React App sets up Jest by default). Key points about Jest: - It is a **test runner** (finds and runs your tests), an **assertion library** (provides functions like `expect` for making assertions), and has built-in support for features like mocking modules/timers and generating coverage reports. It's an all-in-one solution. - Jest is known for its zero config setup (especially for React), parallel test running, and a user-friendly interface with features like snapshots and watch mode. - In React apps, you use Jest to write unit tests and integration tests for your components and logic. A typical test file might be named `MyComponent.test.js` and use Jest's global functions like `test` or `it` and `expect`:

```
import { render, screen, fireEvent } from '@testing-library/react';
import MyComponent from './MyComponent';

test('renders the heading', () => {
  render(<MyComponent />);
  const headingEl = screen.getByRole('heading', { name: /welcome/i });
  expect(headingEl).toBeInTheDocument();
});
```

```
});

test('button click increments counter', () => {
  render(<MyComponent />);
  fireEvent.click(screen.getByText('Increment'));
  expect(screen.getByText('Count: 1')).toBeTruthy();
});
```

In this snippet, Jest is running the tests, and we're using React Testing Library for rendering and interactions (more on that later). The `expect(...).toBeInTheDocument()` style is Jest's assertion combined with some custom matchers from Testing Library/jest-dom.

- Jest also supports **snapshot testing**: you render a UI component to a JSON-like format and save it, then on future test runs it compares to the saved snapshot to detect unintended changes. For example:

```
import renderer from 'react-test-renderer';
it('renders consistently', () => {
  const tree = renderer.create(<MyComponent prop="value" />).toJSON();
  expect(tree).toMatchSnapshot();
});
```

This will create or compare to a snapshot. This technique is useful to ensure UI doesn't change unexpectedly (but snapshots can become large and need review if updated).

- Jest provides powerful **mocking** capabilities: You can mock modules (e.g., mocking an API module to return test data), functions, timers (with fake timers), etc. For instance `jest.mock('axios')` to avoid real HTTP calls and define what it returns.

Overall, Jest is the de facto standard for testing React (and generally JS) due to its integration and features. It focuses on ease of use – e.g., you can run `npm test` and it finds all `*.test.js` files and runs them. It also has a watch mode that runs tests related to changed files and interactive features to re-run failed tests, etc.

Q34. What is React Testing Library (RTL) and how does it differ from Enzyme?

Answer: React Testing Library (RTL) is a testing library that works on top of Jest (or any test runner) to help test React components. Its guiding principle is to test components in a way that is as close as possible to how they are used by end-users – focusing on component output and behavior, not implementation details.

How RTL works: It provides utilities to **render** a component to a “virtual” DOM (JSDOM) and then **query** the rendered output in a human-centric way (like finding elements by their text, labels, roles, etc., similar to how a user would find them on the screen). For example:

```
import { render, screen, fireEvent } from '@testing-library/react';
render(<LoginForm />);
const emailInput = screen.getByLabelText(/email/i);
```

```
fireEvent.change(emailInput, { target: { value: 'test@example.com' } });
expect(screen.getByText(/welcome/i)).toBeInTheDocument();
```

The queries like `getByLabelText`, `getByText`, `getByRole` encourage you to consider accessibility attributes and visible text.

Enzyme: On the other hand, Enzyme (by Airbnb) was an older popular React testing utility (especially before Hooks) that allowed **mounting**, **shallow rendering**, and diving into component internals. It let you inspect component state, call lifecycle methods, etc., in tests. While powerful, it often encouraged testing implementation details (like checking state variables, or the presence of certain child components by component name), which can lead to brittle tests.

Key differences: - **Philosophy:** RTL encourages testing via the component's public interface (the DOM it renders and events), whereas Enzyme allowed testing via internal component instances. For example, in RTL you wouldn't usually assert "state count is 5"; instead you'd assert that you see the text "5" on screen. Enzyme might have you call `wrapper.setState({ count:5 })` and then `expect(wrapper.find('CounterDisplay').text()).toBe("5")` or even access `wrapper.instance().state`. - **APIs:** Enzyme had `shallow` rendering which mounts a component without its children (useful to isolate a component test), and `mount` for full DOM. RTL always does a full render (though you can mock child components if needed). RTL's queries are more like user queries, Enzyme's are more like jQuery (find by selector or component class). - **Future and compatibility:** Enzyme had to be updated for each React version (to handle new APIs, like hooks and new context, etc.), and has lost favor as React Testing Library became the recommended approach in the community. RTL works with React's public API (the DOM and events), so it's less tied to React internals, making it more future-proof.

In summary, **React Testing Library** focuses on testing what the user sees and does (render output and events) which generally leads to more robust tests. **Enzyme** allowed more granular component internals testing but at the risk of being too coupled to implementation. These days, most mid-level devs are expected to know RTL basics and possibly have moved away from Enzyme.

Q35. How would you test a React component that includes asynchronous behavior (like data fetching)?

Answer: Testing a component with asynchronous behavior (such as an effect that fetches data from an API) involves a few things: - **Mocking the async call:** You don't want your test making real network requests. Instead, you can mock the module that performs the fetch (e.g., if using axios or fetch). For fetch, you might use `jest.spyOn(global, 'fetch')` or use `jest.mock('node-fetch')` or a library like `msw` (Mock Service Worker) for a more integrated approach. A simple approach with Jest is to replace `global.fetch` with a mock that returns a resolved promise with the desired data. - **Awaiting the results:** Since the component's effect will likely cause a state change after the data is fetched, you need to wait for the component to finish updating. In RTL, they provide utilities like `findBy...` queries or `waitFor` to wait for changes in the DOM.

For example, imagine a `UserProfile` component that on mount does an API call to load user data and then displays the user's name:

```
// Pseudocode for component:
useEffect(() => {
```

```

    fetch('/api/user/123').then(res => setUser(res.data));
  }, []);
  if (!user) return <div>Loading...</div>;
  return <div>Username: {user.name}</div>;

```

A test for this might be:

```

import { render, screen, waitFor } from '@testing-library/react';
// Suppose the component uses global.fetch or a fetchUser function we can
mock:
global.fetch = jest.fn(() =>
  Promise.resolve({ json: () => Promise.resolve({ name: "Alice" }) })
);

test('loads and displays user', async () => {
  render(<UserProfile />);
  expect(screen.getByText(/Loading/i)).toBeInTheDocument();

  // Wait for the user name to appear
  const nameElement = await screen.findByText(/Username: Alice/i);
  expect(nameElement).toBeInTheDocument();
  // Optionally ensure loading is gone:
  expect(screen.queryByText(/Loading/i)).toBeNull();
});

```

Here, `screen.findByText` is a convenience that returns a promise that resolves when the element is found (or rejects if timeout), so it waits up to a timeout for the text to appear. Alternatively, `waitFor` can be used to wait for arbitrary expectations:

```

await waitFor(() => expect(screen.getByText(/Username: Alice/i)).toBeInTheDocument());

```

Also note we mocked `global.fetch` to return a resolved promise with our fake data (Alice).

If using `axios`, one could `jest.mock('axios')` and set `axios.get.mockResolvedValue({ data: {...} })`.

Important: If your component uses real timers (like `setTimeout` or something), you might need to use `jest.useFakeTimers()` or adjust how you wait. But for `fetch`, usually just waiting works.

Using MSW (Mock Service Worker): A more advanced technique is to use MSW which intercepts actual network calls in a test environment and returns mock responses. This allows testing the full flow without altering the component code to inject a mock.

Finally, ensure you clean up after tests (restore `fetch` mock if needed, or use Jest's `afterEach` to clear mocks).

The key takeaway: to test async, **simulate or mock the async behavior** and **wait for the component to update** in the test, then make assertions on the updated output.

Q36. What is snapshot testing and when would you use it in React?

Answer: Snapshot testing is a technique where you render a UI component (or any serializable value) and capture a “snapshot” of its output. In React, this usually means rendering a component to a JSON structure (using a library like `react-test-renderer`) and saving that JSON to a snapshot file. On subsequent test runs, the new output is compared to the saved snapshot. If it differs, the test fails, alerting you that something changed in the component’s render.

Usage in Jest is straightforward:

```
import renderer from 'react-test-renderer';
it('renders the component correctly', () => {
  const tree = renderer.create(<MyComponent propA="hello" />).toJSON();
  expect(tree).toMatchSnapshot();
});
```

The first time you run this, Jest will save the output (the structure of the rendered DOM) to a file (e.g., `__snapshots__/MyComponent.test.js.snap`). It might look like:

```
exports[`renders the component correctly 1`] = `
<div className="main">
  <span>Hello</span>
  <button>Submit</button>
</div>
`;
```

On future runs, if `MyComponent`’s output changes (say the button text changes to "Send"), the test will fail showing a diff between the new output and the snapshot.

When to use snapshot tests: - To quickly capture the output of a component and ensure it doesn’t unexpectedly change. This is handy for presentational components or components where the output is complex and you want to ensure no regressions in rendering. - For example, if you have a component that renders a large menu or a complex layout, a snapshot can catch if something structural changes. - They are also used for testing that certain props result in expected render output without writing a bunch of manual `expect` statements.

Pros: - Easy to write – you can generate a lot of coverage quickly by snapshotting components. - They can catch accidental changes. For instance, if someone refactors a component and it changes the markup or text slightly, the snapshot test failing will prompt a review of whether that change was intended.

Cons: - Snapshots can become too broad and might fail for legitimate changes. When they fail, you have to manually inspect the diff to see if the change was intended. It can be tempting to just update the snapshot, which then means the test didn’t really validate something purposeful. - Over-reliance on snapshots can make tests less precise. It’s often better to have targeted assertions (e.g., “the title is rendered” or “button is disabled when X”) rather than a huge dump of markup. - Snapshots are best for

UI regression testing where markup/structure is critical, but they shouldn't be the only tests. They complement unit tests by ensuring visual/output consistency.

In short, snapshot testing is a quick way to ensure UI output stays consistent. Use it for components where the structure/markup shouldn't change often. Always review snapshot diffs in code reviews to ensure changes are intentional. It's a valuable tool but should be used alongside more specific tests.

Q37. How would you approach testing a Redux-connected component or testing Redux logic?

Answer: Testing Redux-related code can happen at a couple of levels: testing the Redux **logic** (**reducers, action creators, sagas/thunks**) separately, and testing React components that are connected to Redux.

- **Testing reducers:** Reducers are pure functions, so they're straightforward to test. You can call the reducer with some initial state and an action, and assert that it returns the expected new state. Example:

```
import counterReducer from './counterSlice';
test('counterReducer handles INCREMENT', () => {
  const initialState = { value: 0 };
  const newState = counterReducer(initialState, { type: 'counter/INCREMENT' });
  expect(newState.value).toBe(1);
});
```

Since no side effects in reducers, just verify state transitions.

- **Testing action creators/thunks:**

- For plain action creators (that return an action object), just call them and check the returned object structure.
- For thunks (async actions), you can use a mock store (like using `redux-mock-store` library) or jest mocks for dispatch. One approach: if using Redux Toolkit or thunks, you might export the thunk and call it with a fake dispatch and getState:

```
import { fetchUsers } from './userSlice'; // an async thunk
test('fetchUsers thunk dispatches actions and updates state', async () => {
  const dispatch = jest.fn();
  const getState = () => ({ /* state if needed */ });
  await fetchUsers()(dispatch, getState, undefined);
  // Now check that dispatch was called with request and success actions:
  expect(dispatch).toHaveBeenCalledWith({ type: 'users/fetchUsers/pending', ... });
  expect(dispatch).toHaveBeenCalledWith({ type: 'users/fetchUsers/'
```

```
fulfilled', payload: /*...*/ });  
});
```

Alternatively, use a configured test store with thunk middleware and dispatch the thunk, then check store's state or actions.

- **Testing connected components (integration test):** You can test a component that uses Redux by providing it with a real or test store using `<Provider>`. For example, using React Testing Library:

```
import { Provider } from 'react-redux';  
import { createStore } from 'redux';  
import { render, screen } from '@testing-library/react';  
import Widget from './Widget';  
import rootReducer from './store';  
  
test('Widget displays value from Redux store', () => {  
  // create a test store with initial state  
  const store = createStore(rootReducer, { widget: { value: 42 } });  
  render(  
    <Provider store={store}>  
      <Widget />  
    </Provider>  
  );  
  expect(screen.getByText(/42/)).toBeInTheDocument();  
});
```

This approach mounts the component with a Redux store. You might create a custom render helper in tests that automatically wraps with provider and uses a store (possibly a store configured for tests or even a memory store from Redux Toolkit).

If the component dispatches actions on user events, you can simulate those events and then either spy on `store.dispatch` or check after the event that the expected outcome is in the UI (implying the action and reducer worked). It might be easier to treat it as a black box: for instance, click a button that increments a counter (which dispatches action and redux updates state), then check the new counter value appears.

- **Using Redux with a test store vs mocking Redux:** It's generally better to use an actual store (with possibly a simplified reducer) for integration tests, because that tests the real interaction. Mocking `useSelector` or `useDispatch` could be done, but then you're essentially testing just the component logic in isolation and trusting Redux. For a connected component, providing a store is more realistic.
- **Testing middleware or sagas:** If you use complex middleware or sagas, you might use libraries designed for testing them (like testing sagas by running generator functions step by step). But that might be beyond mid-level detail unless specifically asked.

To sum up, testing Redux involves: - Testing pure functions (reducers, selectors) with straightforward assertions. - Testing async flows (thunks/sagas) with either a mock environment or a real store

environment. - Testing React components that use Redux by rendering them with a `<Provider>` in tests, optionally using a test-specific store or the real reducers with test initial state, then interacting and asserting on the UI.

This ensures both your Redux logic and your component integration with Redux are working as expected.

Q38. How do you test user interactions (like clicking a button, typing in an input) in a React component during unit tests?

Answer: Testing user interactions in React is typically done with a combination of a testing utility (like React Testing Library or Enzyme) and simulating events.

Using React Testing Library (RTL) as an example: - You render the component with `render(<MyComponent />)`. - Use the utilities like `fireEvent` or the newer `userEvent` (from `@testing-library/user-event`) to simulate user actions: - **Clicking a button:** Identify the button element (e.g., `screen.getByRole('button', { name: /submit/i })` or `getByText('Submit')`) and then call `fireEvent.click(button)` or `userEvent.click(button)`. - **Typing in input:** Identify the input (`screen.getByLabelText(/username/i)` for example) and use `fireEvent.change(input, { target: { value: 'new value' } })` or `userEvent.type(input, 'new value')`. The `userEvent.type` is more sophisticated as it triggers events like `keyDown`, `keyUp` in sequence as a real user would. - **Selecting, focusing, etc.:** There are similar events or `userEvent` methods like `selectOptions` for a `<select>`, `tab()` to simulate tab key navigation, `upload` for file input, etc.

After the interaction, you assert the expected outcome: - The component's output might change (e.g., a message appears, the button gets disabled, an API call might be triggered). - If using a state management, maybe the state updates which reflects in UI, which you check. - For example:

```
test('typing updates the controlled input and hitting submit calls
onSubmit', () => {
  const onSubmit = jest.fn();
  render(<LoginForm onSubmit={onSubmit} />);
  userEvent.type(screen.getByLabelText(/email/i), 'test@example.com');
  userEvent.type(screen.getByLabelText(/password/i), 'secret');
  userEvent.click(screen.getByRole('button', { name: /login/i }));
  // Assert that onSubmit was called with correct data
  expect(onSubmit).toHaveBeenCalledWith({ email: 'test@example.com',
    password: 'secret' });
});
```

In this pseudo-test, we simulate typing into two inputs and clicking a login button, then check that the mock `onSubmit` handler was called with expected values.

In case the component doesn't accept a direct callback and instead does internal stuff (like calls `fetch`), you might need to mock those or inspect side effects (like an error message appears on invalid input after clicking a submit, etc.).

Important considerations: - **Async interactions:** If the click triggers an async state change (like a network request then state update), you need to use `waitFor` or `findBy` queries to wait for the result. - **Cleanup:** RTL's `render` comes with automatic cleanup between tests (unmounting components), but if not, ensure you unmount or reset DOM after. - **Event propagation/bubbling:** The testing library events usually simulate bubbling as expected. If you test something like a parent catching an event from child, it should naturally happen if you simulate on the child element.

Testing interactions ensures that from a user's perspective, the component behaves correctly when they do something (click, type, etc.). It's a core part of React component tests.

1 2 3 4 6 10 11 12 13 14 15 23 24 25 26 27 28 29 30 100+ React Interview Questions

Straight from Ex-interviewers | Blog

<https://www.greatfrontend.com/blog/100-react-interview-questions-straight-from-ex-interviewers>

5 7 Top 65+ React JS Interview Questions & Answers for 2025

<https://www.simplilearn.com/tutorials/reactjs-tutorial/reactjs-interview-questions>

8 9 Can senior or mid level react developer list some interview questions since I have a mid-level react interview tomorrow. : r/reactjs

https://www.reddit.com/r/reactjs/comments/12vuood/can_senior_or_mid_level_react_developer_list_some/

16 17 18 19 20 21 22 Top 60+ React Redux Interview Questions And Answers -2025 - GeeksforGeeks

<https://www.geeksforgeeks.org/reactjs/react-redux-interview-questions-and-answers/>

31 React Router Interview Questions And Answers - GeeksforGeeks

<https://www.geeksforgeeks.org/reactjs/top-react-router-interview-questions/>